SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1513

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**

Question Count: 5

Code of Conduct

I __J.P.BADRI(192524301)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: ____ J.P.B _____

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.

   Design a cloud architecture using scalability and caching techniques.

   Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.

   They face challenges in maintaining consistency and managing deployments.

   Propose a solution using container orchestration and multi-cloud strategies.

   Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.

   Design a solution using cloud-native backup, replication, and versioning.
   Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.

   Design a hybrid architecture that ensures secure communication between environments.

Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.

   The system must continue operating even if one region goes down.

   Propose a multi-region deployment strategy in the cloud.

   Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and	Reasonable justification	Weak justification	No useful justification

		technically strong			
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

1. Cloud Architecture for a Video Streaming Platform

Introduction

A media company serving millions of concurrent users requires a scalable, high-performance, and cost-effective cloud architecture. The system must deliver video content with minimal buffering, low latency, and high availability.

Proposed Architecture

The company can use a combination of cloud computing, content delivery networks (CDNs), virtual machines, containers, load balancing, and auto-scaling.

Architecture flow:

Users → DNS Routing → CDN Edge Servers → Load Balancer → Auto-Scaling Application Servers → Video Storage and Database

Scalability and Caching Techniques

1. **Content Delivery Network (CDN):** A CDN stores copies of videos on geographically distributed edge servers. Users receive content from nearby servers, reducing latency and buffering.
2. **Object Storage:** Store video files in scalable cloud object storage. Video content can be encoded into multiple resolutions to support adaptive bitrate streaming.
3. **Caching:** Cache frequently accessed video segments, thumbnails, and metadata using edge caches and in-memory caching systems. This reduces repeated requests to the origin servers.
4. **Horizontal Scaling:** Add more application server instances when the number of concurrent users increases.
5. **Virtualization:** Use virtual machines or containers to isolate application components and allocate computing resources efficiently.

Role of Load Balancing and Auto-Scaling

Load balancing distributes incoming requests across multiple healthy application servers. Health checks help prevent traffic from being sent to failed instances.

Auto-scaling automatically increases or decreases the number of server instances based on metrics such as CPU utilization, request rate, and concurrent connections. Predictive scaling can prepare resources for expected traffic peaks.

Together, these techniques prevent server overload, improve response time, and maintain smooth streaming during sudden traffic increases.

Cost Optimization Strategies

- Use pay-as-you-go cloud resources and scale down unused capacity during low-demand periods.
- Apply CDN caching to reduce origin-server processing and data-transfer costs.
- Use reserved capacity or savings plans for predictable workloads and spot instances for fault-tolerant background tasks.
- Apply storage lifecycle policies to move old or rarely accessed videos to lower-cost storage tiers.
- Monitor bandwidth, storage, and compute usage to identify unnecessary expenses.

Performance and Security Issues

Potential issues include CDN cache misses, network congestion, overloaded servers, unauthorized content access, and denial-of-service attacks. These can be addressed through multi-CDN strategies where appropriate, monitoring, rate limiting, encryption, access controls, and distributed denial-of-service protection.

Conclusion

A CDN-based architecture combined with load balancing, auto-scaling, caching, and scalable storage provides reliable streaming performance. Cost monitoring and automated resource management help the company serve millions of users economically.

2. Container Orchestration and Multi-Cloud Deployment for a Fintech Startup

Introduction

A fintech startup deploying containerized applications across multiple cloud providers must maintain consistent environments, secure deployments, reliable service communication, and efficient scaling.

Proposed Solution

Use **Docker containers** to package applications and **Kubernetes** to orchestrate container deployment across cloud environments.

Architecture flow:

Source Code → CI/CD Pipeline → Container Registry → Kubernetes Clusters in Cloud A and Cloud B → Service Discovery and Load Balancing → Monitoring System

Container Orchestration

Kubernetes manages container deployment, scheduling, scaling, and recovery.

1. **Standardized containers:** Package the application, libraries, and dependencies into container images to maintain consistency across providers.
2. **Declarative configuration:** Use Kubernetes manifests or Helm charts to define deployments, services, resource limits, and configuration.
3. **Automated deployment:** A CI/CD pipeline tests and scans images before deploying them to cloud clusters.
4. **Self-healing:** Kubernetes restarts failed containers and replaces unhealthy instances.
5. **Configuration management:** Use externalized configuration and secret-management services rather than embedding credentials in container images.

Multi-Cloud Strategy

Deploy Kubernetes clusters in two cloud providers and manage them through a centralized deployment process. Use infrastructure as code, such as Terraform, to standardize networking and infrastructure provisioning.

Applications should be packaged consistently, while provider-specific services are isolated behind common interfaces wherever practical.

Service Discovery and Scaling

- **Service discovery:** Kubernetes Services and DNS provide stable names for communicating with application components inside each cluster.
- **Cross-cluster communication:** Use a service mesh, secure API gateway, or private network connectivity for communication between cloud environments.
- **Horizontal scaling:** Kubernetes Horizontal Pod Autoscaler adjusts the number of application pods based on CPU, memory, or custom metrics.
- **Cluster scaling:** Cluster autoscaling adds or removes worker nodes according to workload demand.
- **Traffic management:** Global load balancing directs users to healthy endpoints and can shift traffic away from an unavailable cloud.

Benefits of Multi-Cloud Deployment

- **High availability:** Failure of one provider may not interrupt the entire service.
- **Reduced vendor dependency:** Applications can be distributed across providers.

- **Geographical flexibility:** Deploy workloads closer to customers.
- **Service selection:** Choose providers based on performance, compliance, and cost.

Risks and Mitigation

Risk	Mitigation
Different cloud APIs and services	Standardize deployment with Kubernetes and infrastructure as code
Increased operational complexity	Use centralized monitoring and automation
Data inconsistency	Define replication rules and transaction-consistency requirements
Security misconfiguration	Apply least-privilege IAM, encryption, and regular audits
Higher network and operational costs	Monitor cross-cloud traffic and simplify unnecessary data movement

For fintech applications, financial transactions must also use idempotency controls, audit logging, and appropriate consistency mechanisms to prevent duplicate or inconsistent transactions.

Conclusion

Kubernetes, container registries, CI/CD pipelines, and infrastructure as code provide consistent deployments across cloud providers. Multi-cloud deployment improves resilience and flexibility, but requires careful management of security, data consistency, and operational complexity.

3. Cloud Backup and Disaster Recovery for an E-Learning Platform

Introduction

Accidental deletion of educational materials, student records, or course data can interrupt learning and cause permanent data loss. A cloud-native backup and disaster recovery strategy must protect data and enable timely restoration.

Proposed Architecture

Use object storage with versioning, automated backups, cross-region replication, database point-in-time recovery, and an isolated backup account or environment.

Architecture flow:

Production Application → Primary Database and Object Storage → Versioning and Automated Backups → Cross-Region Replication → Secondary Recovery Environment

Backup, Replication, and Versioning

1. **Automated backups:** Schedule regular database snapshots and backups of important application data.
2. **Object versioning:** Enable versioning so deleted or overwritten files can be recovered from earlier versions.
3. **Cross-region replication:** Replicate critical data to a separate geographical region to protect against regional failures.
4. **Point-in-time recovery:** Maintain database transaction logs so the database can be restored to a selected time before an error occurred.
5. **Immutable backups:** Protect backup copies using retention locks or immutable storage to reduce the risk of malicious deletion.
6. **Access control:** Restrict backup deletion permissions and use a separate backup account where possible.

Replication alone is not sufficient because accidental deletions or corrupted data may also be replicated. Independent backup versions and retention policies are therefore essential.

Recovery Time Objective (RTO) and Recovery Point Objective (RPO)

Recovery Time Objective (RTO): The maximum acceptable duration for restoring the service after a failure.

Recovery Point Objective (RPO): The maximum acceptable amount of data loss measured in time.

For example, the organization could set an RTO of 1 hour and an RPO of 15 minutes. These are illustrative targets and must be validated against business requirements and tested recovery performance.

- **Achieving RTO:** Maintain documented recovery procedures, automated infrastructure deployment, preconfigured standby resources, and tested restoration processes.
- **Achieving RPO:** Perform frequent backups, replicate critical data continuously or frequently, and maintain transaction logs.

- **Validation:** Conduct periodic recovery tests to confirm that the actual recovery time and recoverable data meet the targets.

Business Continuity Steps

1. Detect the incident and identify the affected resources.
2. Stop further damage by restricting compromised access or isolating affected systems.
3. Activate the disaster recovery plan and determine the recovery point.
4. Restore data from a verified backup or fail over to a healthy replica.
5. Validate database integrity, user accounts, course content, and application functionality.
6. Redirect users to the recovered service and communicate service status.
7. Monitor the restored environment for errors and investigate the root cause.
8. Document the incident and improve the recovery plan.

Performance and Security Considerations

Frequent backups can increase storage and network costs. Encryption, least-privilege access, immutable retention, backup integrity checks, and separation of administrative permissions help protect recovery copies. Recovery tests also prevent failures caused by incomplete or outdated backups.

Conclusion

A combination of versioning, automated backups, cross-region replication, immutable storage, and point-in-time recovery protects the e-learning platform against accidental deletion and major failures. Regular testing ensures that recovery objectives are achievable.

4. Secure Hybrid Cloud Architecture for a Government Agency

Introduction

A government agency may need to keep classified or regulated applications on-premises while using public cloud resources for less-sensitive workloads. A hybrid cloud architecture must provide secure connectivity, centralized identity management, and appropriate data protection.

Proposed Architecture

Architecture flow:

On-Premises Data Center ↔ Encrypted VPN or Dedicated Private Connection ↔ Cloud Network

- **On-premises environment:** Hosts sensitive applications, regulated databases, and workloads that must remain within government-controlled infrastructure.
- **Public cloud:** Hosts approved non-sensitive applications, web front ends, analytics, and scalable services.
- **Secure connectivity:** Connect the environments through an encrypted site-to-site VPN or a dedicated private connection protected by appropriate encryption and network controls.
- **Network segmentation:** Separate sensitive systems, application tiers, and public-facing services using subnets, firewalls, and access-control rules.

Identity and Access Management

1. **Centralized identity:** Integrate on-premises directory services with cloud identity management using secure federation.
2. **Multi-factor authentication (MFA):** Require MFA for administrators and other sensitive accounts.
3. **Role-based access control (RBAC):** Grant permissions according to job responsibilities and the principle of least privilege.
4. **Privileged access management:** Use controlled, audited administrative access and temporary elevated permissions.
5. **Audit logging:** Record authentication, configuration changes, data access, and administrative activities in a protected logging system.

Encryption and Data Protection

- **Data in transit:** Use TLS for application communications and IPsec for VPN connections where appropriate.
- **Data at rest:** Encrypt databases, storage volumes, backups, and object storage.
- **Key management:** Use a centralized or federated key-management approach with strict permissions, key rotation, and auditing.
- **Data classification:** Classify information according to sensitivity and apply corresponding storage, access, and retention rules.
- **Data residency:** Ensure that regulated data remains in approved locations and is not replicated to unauthorized regions.

Encryption keys and secrets should be protected separately from the data they secure.

Challenges in Hybrid Cloud Management

Challenge	Solution
Complex network configuration	Use standardized network designs and centralized policy management
Identity synchronization issues	Test federation and define secure account recovery procedures
Compliance requirements	Apply continuous compliance checks and regular audits
Latency between environments	Keep latency-sensitive applications and databases close together
Inconsistent security controls	Use common security policies and centralized monitoring
Operational complexity	Automate provisioning and configuration using infrastructure as code

Performance and Security Analysis

Hybrid environments can experience network latency, bandwidth limitations, and connectivity failures. Redundant network connections and performance monitoring improve reliability. Misconfigured access rules or identity federation can expose sensitive resources, so access reviews, encryption, segmentation, and security testing are necessary.

Conclusion

A secure hybrid cloud architecture allows the agency to retain sensitive workloads on-premises while using public cloud resources for approved services. Encrypted connectivity, federated identity, least-privilege access, and centralized auditing support security and compliance.

5. Multi-Region Deployment for a Highly Available SaaS Application

Introduction

A SaaS provider must ensure that its application remains available even if an entire cloud region becomes unavailable. A multi-region architecture distributes application components and data across geographically separate regions.

Proposed Architecture

Use an active-active or active-passive deployment across at least two cloud regions.

Architecture flow:

Users → Global DNS or Traffic Manager → Region A and Region B → Regional Load Balancers → Application Instances → Replicated Database and Storage

Multi-Region Deployment Strategy

1. **Deploy in multiple regions:** Run application instances in at least two independent regions.
2. **Use regional load balancers:** Distribute requests across healthy application instances within each region.
3. **Configure global traffic management:** Route users to healthy regions using health checks and suitable latency- or priority-based routing.
4. **Automate infrastructure:** Use infrastructure as code and deployment pipelines to keep environments consistent.
5. **Avoid single points of failure:** Ensure that critical dependencies, authentication services, networking, and storage have suitable redundancy.

Active-active deployment: Both regions serve traffic simultaneously. This can improve utilization and availability, but requires careful management of concurrent writes and data consistency.

Active-passive deployment: One region handles normal traffic while another remains ready to take over. It can simplify data consistency but may require more time to activate standby resources.

Data Replication and Failover

- **Database replication:** Replicate data to the secondary region using synchronous replication when latency and infrastructure permit, or asynchronous replication when performance and geographical distance make it more appropriate.
- **Consistency management:** Define how concurrent updates, conflicts, and transactions are handled. Use a database architecture designed for the required consistency guarantees.
- **Storage replication:** Replicate important files, configuration, and backups to the secondary region.
- **Health checks:** Monitor application endpoints, database health, and critical dependencies.
- **Automated failover:** Route traffic to the healthy region when the primary region fails.

- **Failback:** After the failed region is restored, validate data consistency, synchronize missing updates, and return traffic gradually.

Asynchronous replication can result in some recent writes being lost during a sudden failure. Synchronous replication can reduce that risk but may increase latency and reduce write availability during network disruptions.

Monitoring and Alerting Strategies

Monitor the following metrics:

- Application availability, response time, and error rate.
- CPU, memory, storage, and network utilization.
- Database replication lag and connection failures.
- Failed requests, queue backlogs, and authentication errors.
- Regional health-check failures and failover status.

Use centralized dashboards, logs, distributed tracing, and automated alerts. Configure alerts for service-level objective violations, unusual error spikes, replication delays, and resource exhaustion. Route critical alerts to the responsible operations team and define escalation procedures.

Regularly test regional failure scenarios to confirm that traffic routing, data recovery, and application dependencies work as expected.

Performance, Security, and Cost Considerations

Multi-region deployment improves availability but increases infrastructure, replication, monitoring, and data-transfer costs. Security controls should be consistent across regions, including encryption, IAM policies, secrets management, and audit logging. Regional data residency rules must also be considered.

Conclusion

A multi-region architecture with redundant application instances, suitable database replication, global traffic management, and automated failover reduces the impact of regional outages. Continuous monitoring and disaster recovery testing help maintain reliable SaaS operations.